

SignBridge

Plateforme de Traduction de la Langue des Signes par Intelligence Artificielle

Établissement : ENSET de Douala

Projet : SignBridge — Groupe RYDI

Date : 27 avril 2026

Technologies : Python 3.10 · Django 5.2 · MediaPipe · scikit-learn · TensorFlow · Three.js

Table des Matières

Table des Matières	2
1. Présentation du Projet SignBridge	4
1.1 Objectifs.....	4
1.2 Capture d'écran — Page d'accueil.....	4
1.3 Capture d'écran — Interface principale	4
2. Technologies Utilisées et leur Rôle	6
2.1 Backend — Python / Django	6
2.2 Intelligence Artificielle et Machine Learning	6
2.3 Frontend — Interface Utilisateur	6
2.4 Base de données.....	7
3. Outils Matériels et Logiciels.....	8
3.1 Environnement de développement.....	8
3.2 Matériel requis	8
3.3 Datasets utilisés	8
4. Méthodes d'Intelligence Artificielle.....	9
4.1 Extraction de landmarks — MediaPipe Hands	9
Principe de fonctionnement	9
Les 21 points landmarks MediaPipe	9
4.2 Classification statique — Random Forest.....	9
Caractéristiques du modèle	9
4.3 Classification dynamique — LSTM (Long Short-Term Memory)	9
4.4 Pipeline de reconnaissance en temps réel	10
5. Modules Développés	11
5.1 Module Backend — core/	11
5.2 Module ML — ml_engine/	11
5.3 Module Frontend — static/js/	11
5.4 Commandes de gestion (manage.py)	12
6. Procédure de Création des Avatars 3D	13
6.1 Architecture de l'avatar — Hand3DRenderer	13
Composition géométrique.....	13
Connexions topologiques.....	13
6.2 Pipeline des poses — de la source aux coordonnées 3D.....	13
Étape 1 — Définition des poses (generate_asl_training.py)	13
Étape 2 — Augmentation des données	13
Étape 3 — Calcul de la pose moyenne (API)	13
Étape 4 — Rendu Three.js (hand3d_renderer.js).....	13
6.3 Captures — Mode Avatar 3D	14
7. Reconnaissance des Signes — Fonctionnement Détaillé.....	15
7.1 Mode Caméra (temps réel)	15
7.2 Mode Vidéo (upload).....	15
7.3 Paramètres de stabilisation.....	15
8. Entraînement du Modèle ML.....	17

8.1	Commande d'entraînement	17
8.2	Pipeline d'entraînement — Random Forest	17
8.3	Pipeline d'entraînement — LSTM	17
8.4	Entraînement via l'interface admin	17
8.5	Composition du dataset (38 087 samples)	17
9.	Configurations du Projet.....	19
9.1	Fichier .env (variables d'environnement).....	19
9.2	Configuration Django (config/settings.py)	19
9.3	Configuration ASGI (config/asgi.py)	19
9.4	Démarrage du serveur	19
9.5	Paramètres du renderer 3D (hand3d_renderer.js)	19
10.	Diagrammes UML	21
10.1	Diagramme de Cas d'Utilisation	21
10.2	Diagramme de Séquence — Reconnaissance Webcam	21
10.3	Diagramme de Séquence — Texte vers Avatar 3D	22
10.4	Diagramme de Classes	24
10.5	Schéma Synoptique de l'Architecture	24
11.	Dictionnaire des Signes	26
11.1	Code PlantUML des diagrammes	26
12.	Conclusion	27

1. Présentation du Projet SignBridge

SignBridge est une application web full-stack de traduction bidirectionnelle de la langue des signes. Elle permet à des utilisateurs entendants et malentendants de communiquer à travers trois modes complémentaires : la traduction de texte vers signe (avatar 2D ou main 3D), la reconnaissance de signe en temps réel via webcam, et la traduction d'une vidéo de signe vers du texte français.

Le projet a été développé dans le cadre d'un cursus de niveau 3 à l'ENSET de Douala, groupe RYDI. Il constitue un prototype fonctionnel démontrant l'intégration de techniques d'intelligence artificielle modernes dans une interface web accessible.

1.1 Objectifs

- Permettre la traduction Texte → Signe ASL/LSF via un avatar 2D et une main 3D procédurale
- Reconnaître les signes en temps réel depuis la webcam de l'utilisateur
- Offrir un dictionnaire visuel des signes A-Z et 0-9
- Fournir une interface d'administration pour gérer le dataset et entraîner le modèle ML

1.2 Capture d'écran — Page d'accueil

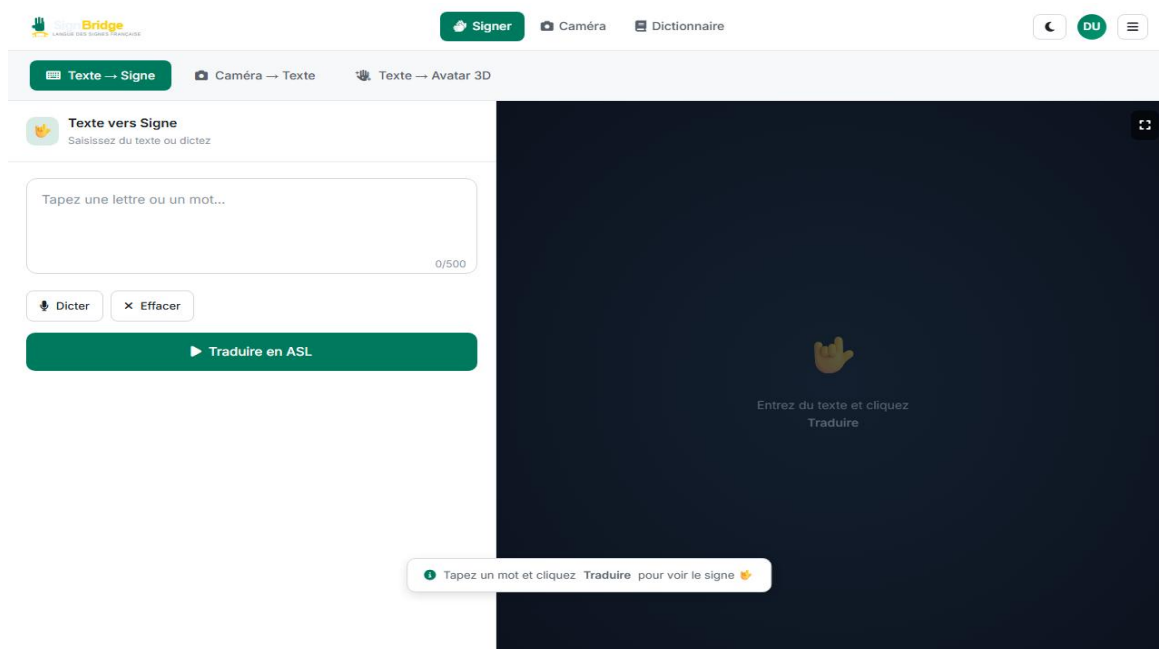


Figure 1 : Page d'accueil de SignBridge

1.3 Capture d'écran — Interface principale

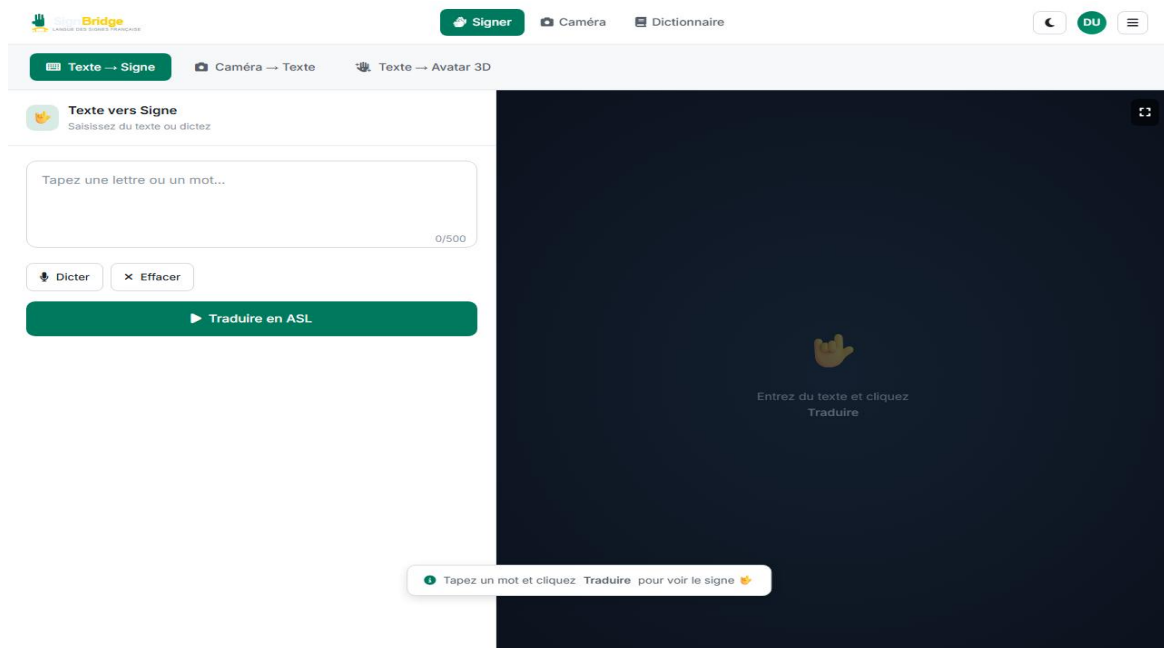


Figure 2 : Interface principale — Mode Texte → Signe

2. Technologies Utilisées et leur Rôle

2.1 Backend — Python / Django

Technologie	Version	Rôle dans SignBridge
Django	5.2	Framework web principal — routing HTTP, ORM, gestion des sessions, templates Jinja2
Django Channels	4.1.0	Extension Django pour WebSocket ASGI — communication temps réel pour la reconnaissance gestuelle
Daphne	4.1.0	Serveur ASGI — sert simultanément HTTP et WebSocket (protocoles asynchrones)
WhiteNoise	6.6.0	Sert les fichiers statiques (CSS, JS, images, modèles GLB) directement depuis Django sans Nginx
python-dotenv	1.0.1	Chargement des variables d'environnement depuis le fichier .env (SECRET_KEY, DEBUG, etc.)
Gunicorn	21.2.0	Serveur WSGI de production pour le déploiement

2.2 Intelligence Artificielle et Machine Learning

Bibliothèque	Version	Rôle
MediaPipe	0.10.11	Extraction des 21 landmarks de la main (x, y, z) en temps réel — cœur de la détection gestuelle
scikit-learn	1.4.2	Random Forest Classifier — modèle principal de classification statique des signes (36 classes A-Z + 0-9)
TensorFlow	2.15.0	LSTM (Long Short-Term Memory) — classification de séquences dynamiques de signes (30 frames)
NumPy	1.26.4	Manipulation des tableaux de landmarks, normalisation, calcul des poses moyennes pour l'avatar
joblib	1.3.2	Sérialisation/désérialisation du modèle Random Forest (rf_model.pkl, 769 Mo)
OpenCV	4.9.0	Lecture et décodage des fichiers vidéo pour l'extraction frame-par-frame

2.3 Frontend — Interface Utilisateur

Technologie	Version	Rôle
Tailwind CSS	3.4.1	Framework CSS utility-first — styles compilés avec npx, thème sombre/clair

Technologie	Version	Rôle
Three.js	r160.1	Rendu 3D WebGL — visualisation de la main procédurale (21 sphères + 26 cylindres) avec animations fluides
MediaPipe JS	0.10.x	Détection des landmarks main côté client via WebAssembly — traitement vidéo en temps réel dans le navigateur
Web Speech API	Natif	Reconnaissance vocale — dictée du texte à signer dans l'interface
WebSocket API	Natif	Communication bidirectionnelle avec le serveur Django Channels pour l'envoi de landmarks
Canvas 2D API	Natif	Rendu des images de signes ASL avec animation flottante (HandSignRenderer)

2.4 Base de données

Le projet utilise SQLite comme base de données relationnelle embarquée. À la date d'analyse, la base db.sqlite3 occupe 58 Mo et contient notamment :

- 38 087 DatasetSample (landmarks de mains pour 36 signes)
- 36 SignDictionary (alphabet A-Z + chiffres 0-9)
- Comptes utilisateurs, sessions de traduction, versions du modèle ML

3. Outils Matériels et Logiciels

3.1 Environnement de développement

Outil	Description
Python 3.10+	Langage backend — compatible avec MediaPipe 0.10.x et TensorFlow 2.15
Node.js / npm	Compilation du CSS Tailwind (npm run build:css) — seule dépendance JS côté dev
VS Code / IDE	Environnement de développement principal
Git	Gestion de versions du code source
SQLite Browser	Inspection et requêtes directes sur db.sqlite3

3.2 Matériel requis

- Webcam HD (720p minimum) — pour la reconnaissance gestuelle en temps réel
- RAM ≥ 4 Go — le modèle Random Forest (rf_model.pkl) charge 769 Mo en mémoire au démarrage
- Navigateur moderne avec WebGL 2.0 — pour le rendu Three.js (Chrome, Firefox, Edge récents)
- Processeur multi-cœur — TensorFlow peut utiliser les cœurs CPU pour l'inférence LSTM

3.3 Datasets utilisés

Dataset	Source	Contenu
mediapipe-processed-asl-dataset	Kaggle (vignonantoine)	20 834 landmarks A-Z et 0-9 extraits de vidéos ASL (coordonnées 2D pour les lettres)
American Sign Language Digits Dataset	Kaggle (Images JPEG)	2 853 landmarks 0-9 extraits via MediaPipe — vraies coordonnées 3D
Données synthétiques ASL_POSES	Génération interne	14 400 samples générés par augmentation mathématique (bruit, rotation, échelle) à partir de poses hardcodées

4. Méthodes d'Intelligence Artificielle

4.1 Extraction de landmarks — MediaPipe Hands

MediaPipe Hands est la brique fondamentale de la détection gestuelle dans SignBridge. Développé par Google, il détecte et suit la main en temps réel à partir d'une image ou d'un flux vidéo.

Principe de fonctionnement

- Détection de paume : un modèle CNN léger (MobileNet) localise d'abord la paume dans l'image complète
- Régression des landmarks : un second réseau prédit la position précise des 21 points anatomiques de la main
- Coordonnées retournées : x, y normalisés [0-1] dans l'espace image, z (profondeur relative) dans la même échelle
- Traitement : 12-30 fps côté navigateur (WebAssembly), ou en Python côté serveur pour les vidéos

Les 21 points landmarks MediaPipe

Chaque main est représentée par 21 landmarks numérotés de 0 à 20 :

- Point 0 : Poignet (wrist) — point de référence pour la normalisation
- Points 1-4 : Pouce (CMC, MCP, IP, Tip)
- Points 5-8 : Index (MCP, PIP, DIP, Tip)
- Points 9-12 : Majeur (MCP, PIP, DIP, Tip)
- Points 13-16 : Annulaire (MCP, PIP, DIP, Tip)
- Points 17-20 : Auriculaire (MCP, PIP, DIP, Tip)

Le vecteur de features est de dimension 63 (21 points $\times$ 3 coordonnées x,y,z). Avant classification, il est normalisé par centrage sur le poignet (point 0 soustrait de tous les points) pour rendre la prédiction invariante à la position de la main dans l'image.

4.2 Classification statique — Random Forest

Le modèle principal de SignBridge est un Random Forest Classifier de scikit-learn, entraîné pour classer 36 signes distincts (lettres A-Z et chiffres 0-9).

Caractéristiques du modèle

Paramètre	Valeur
Algorithme	Random Forest Classifier (ensemble de 100 arbres de décision)
Classes	36 (A à Z + 0 à 9)
Features d'entrée	63 floats (21 landmarks $\times$ 3 coordonnées)
Taille du modèle	769 Mo (rf_model.pkl sérialisé par joblib)
Précision	83% (accuracy sur jeu de test)
Fichier	signbridge_model/rf_model.pkl
Encodeur	LabelEncoder — signbridge_model/label_encoder.pkl
Chargement	Singleton au démarrage Django (AppConfig.ready() en thread daemon)

4.3 Classification dynamique — LSTM (Long Short-Term Memory)

Pour la reconnaissance de signes dynamiques (nécessitant un mouvement), SignBridge intègre un réseau LSTM implémenté avec TensorFlow/Keras.

- Entrée : séquence de 30 frames consécutives de landmarks ($30 \times 63 = 1890$ valeurs)
- Architecture : couches LSTM empilées suivies de couches Dense avec activation softmax
- Segmentation : la vidéo est découpée en segments de 30 frames avec chevauchement de 15 frames

- Seuil de confiance : seulement les prédictions avec confiance > 0.6 sont retenues
- Fichiers : lstm_model.keras (prioritaire) et lstm_model.h5 (fallback)

4.4 Pipeline de reconnaissance en temps réel

Le flux complet de reconnaissance via webcam suit cette pipeline :

1. Frame vidéo capturée par le navigateur (getUserMedia)
2. MediaPipe JS détecte les 21 landmarks (WASM, ~12 fps)
3. Les coordonnées sont envoyées via WebSocket au serveur Django Channels
4. SignRecognitionConsumer normalise les landmarks (centrage poignet)
5. GestureClassifier.predict_static() appelle le Random Forest
6. Le label et la confiance sont retournés au navigateur via WebSocket
7. Mécanisme de stabilisation : 3 frames consécutives identiques requises avant validation (STABLE_NEEDED = 3)
8. Seuil de confiance : 0.22 normal, 0.11 si signe stable depuis 6 frames

5. Modules Développés

5.1 Module Backend — core/

Fichier	Description
models.py	5 modèles ORM : CustomUser (email login), SignDictionary, DatasetSample (landmarks), TranslationSession, MLModelVersion
views.py	18 vues : 7 pages HTML + 7 endpoints API JSON (translate, dictionary, ml/status, stats, train, avatar-landmarks, video-to-text)
consumers.py	SignRecognitionConsumer — WebSocket ASGI : reçoit landmarks JSON, appelle GestureClassifier, retourne prédiction
forms.py	InscriptionForm + ConnexionForm (authentification par email)
decorators.py	@admin_required — décorateur de protection des vues admin
apps.py	AppConfig.ready() — chargement du modèle RF en thread daemon au démarrage
routing.py	Routage WebSocket : ws/sign-recognition/ → SignRecognitionConsumer
urls.py	14 routes URL (pages publiques, pages protégées, panel admin, API)

5.2 Module ML — ml_engine/

Fichier	Description
gesture_classifier.py	Singleton GestureClassifier — charge RF + LSTM, expose predict_static() et predict_dynamic()
landmark_extractor.py	LandmarkExtractor — wrapper MediaPipe Python, extrait landmarks depuis frame ou fichier vidéo
model_trainer.py	ModelTrainer — entraîne RF et LSTM depuis les DatasetSample en base, sauvegarde les modèles
dataset_manager.py	Chargement du dataset depuis la DB, sauvegarde de nouveaux DatasetSample
video_processor.py	VideoProcessor — traite une vidéo complète (extraction → segmentation → classification → texte)

5.3 Module Frontend — static/js/

Fichier JS	Rôle
webcam_client.js	Client WebSocket + initialisation MediaPipe — gère la boucle de capture et l'envoi des landmarks
hand3d_renderer.js	Renderer Three.js — construit une main 3D procédurale (21 sphères + 26 cylindres), interpole les poses (tween ease-in-out)
hand_sign_renderer.js	Renderer Canvas 2D — affiche les images JPG des signes avec animation flottante (breathing)
avatar_player.js	AvatarPlayer — joue une séquence de signes avec gestion race-condition par compteur de génération

Fichier JS	Rôle
speech_input.js	SpeechInput — wrapper Web Speech API pour la saisie vocale du texte à traduire
video_upload.js	VideoUpload — upload vidéo XHR avec validation taille/format, barre de progression

5.4 Commandes de gestion (manage.py)

- setup_demo_data — crée 36 signes + comptes admin/démo en base
- train_model --algo all — entraîne RF et/ou LSTM depuis les DatasetSample
- extract_landmarks — extrait landmarks depuis vidéos brutes
- extract_from_images — extrait landmarks depuis images JPEG
- import_kaggle_asl — importe le dataset ASL Kaggle (CSV landmarks)
- import_landmarks_json — importe un fichier JSON de landmarks
- generate_asl_training — génère 14 400 DatasetSample synthétiques (A-Z + 0-9)

6. Procédure de Création des Avatars 3D

6.1 Architecture de l'avatar — Hand3DRenderer

L'avatar 3D de SignBridge est une main procédurale construite entièrement en Three.js, sans mesh 3D externe ni fichier de modèle. Elle est composée de primitives géométriques positionnées selon les landmarks MediaPipe.

Composition géométrique

- 21 sphères (SphereGeometry) — une par articulation, taille variable : 0.040 pour le poignet, 0.026 pour les extrémités, 0.021 pour les nœuds intermédiaires
- 26 cylindres (CylinderGeometry) — un par connexion osseuse, reliant les articulations selon la topologie MediaPipe
- Couleurs par doigt : vert (poignet), ambre (pouce), bleu (index), violet (majeur), rose (annulaire), cyan (auriculaire)
- Matériaux MeshPhong avec shininess et émissivité légère pour un rendu 3D réaliste

Connexions topologiques

```
[0,1],[1,2],[2,3],[3,4]    // Pouce (4 segments)
[0,5],[5,6],[6,7],[7,8]    // Index (4 segments)
[0,9],[9,10],[10,11],[11,12] // Majeur (4 segments)
[0,13],[13,14],[14,15],[15,16] // Annulaire (4 segments)
[0,17],[17,18],[18,19],[19,20] // Auriculaire (4 segments)
[5,9],[9,13],[13,17]       // Paume (3 connexions transverses)
```

6.2 Pipeline des poses — de la source aux coordonnées 3D

Les poses de l'avatar sont générées via la commande `generate_asl_training` puis stockées en base de données. L'API `/api/avatar-landmarks/` les calcule et les retourne au navigateur.

Étape 1 — Définition des poses (`generate_asl_training.py`)

Chaque signe A-Z et 0-9 est défini par un dictionnaire `ASL_POSES` contenant les positions 3D des 21 landmarks pour la pose de référence. Ces coordonnées sont définies manuellement comme des vecteurs NumPy simulant une vue depuis une caméra frontale.

Étape 2 — Augmentation des données

- 300 variantes générées par signe par la fonction `augment()`
- Bruit gaussien $\sigma = 2.6\%$ sur toutes les coordonnées
- Rotation 2D aléatoire $\pm 15^\circ$ dans le plan XY
- Mise à l'échelle $\pm 12\%$ (zoom in/out)
- Translation $\pm 4\%$ (déplacement de la main)
- Normalisation par la distance poignet→MCP-majeur
- Résultat : 14 400 `DatasetSample` avec `source='GENERATED'`

Étape 3 — Calcul de la pose moyenne (API)

- GET `/api/avatar-landmarks/` récupère les 40 premiers samples par signe
- Moyenne NumPy des vecteurs de 63 floats → reshape (21, 3)
- Centrage sur le poignet : `mean = mean - mean[0]`
- Retourne un JSON `{A: [{x,y,z}×21], B: [...], ..., 9: [...]}`

Étape 4 — Rendu Three.js (`hand3d_renderer.js`)

- Scale $S = 0.75$ appliqué aux coordonnées (calibré pour tenir dans le cadre caméra)
- Inversion de l'axe Y : Three.js $Y = -\text{landmark.y} \times S$ (MediaPipe Y pointe vers le bas, Three.js vers le haut)
- Position caméra : (0, 0.25, 2.40), regardant (0, 0.25, 0)
- Interpolation tween ease-in-out entre deux poses (450 ms de transition)

- Boucle requestAnimationFrame à ~60 fps

6.3 Captures — Mode Avatar 3D

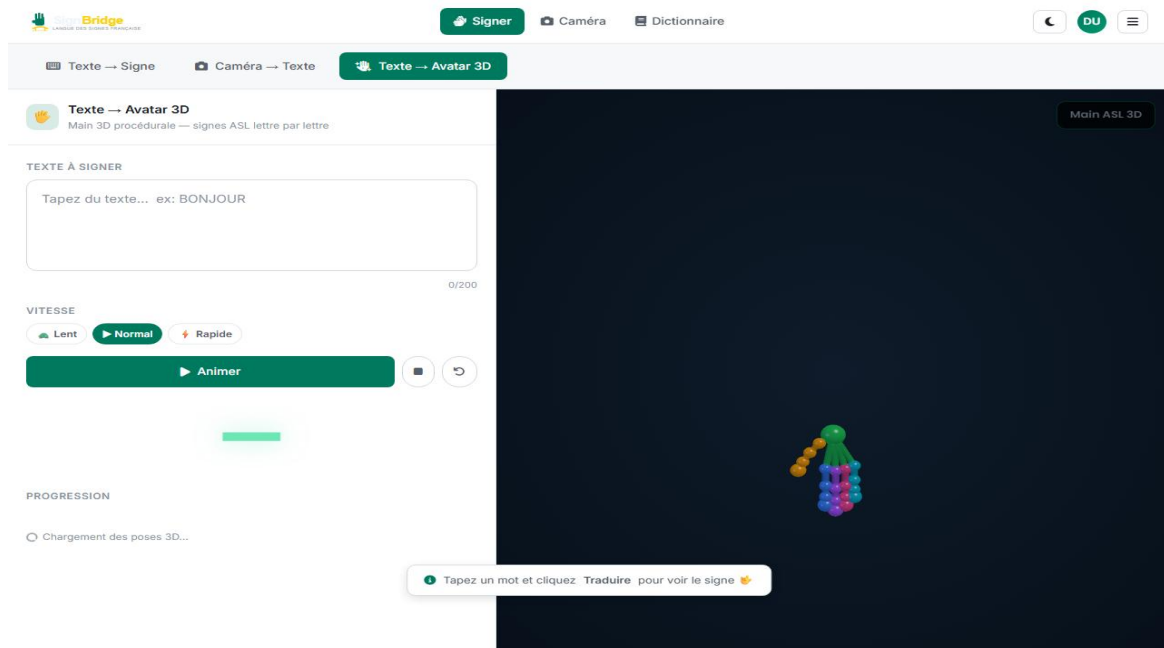


Figure 3 : Mode Avatar 3D — main procédurale Three.js

7. Reconnaissance des Signes — Fonctionnement Détaillé

7.1 Mode Caméra (temps réel)

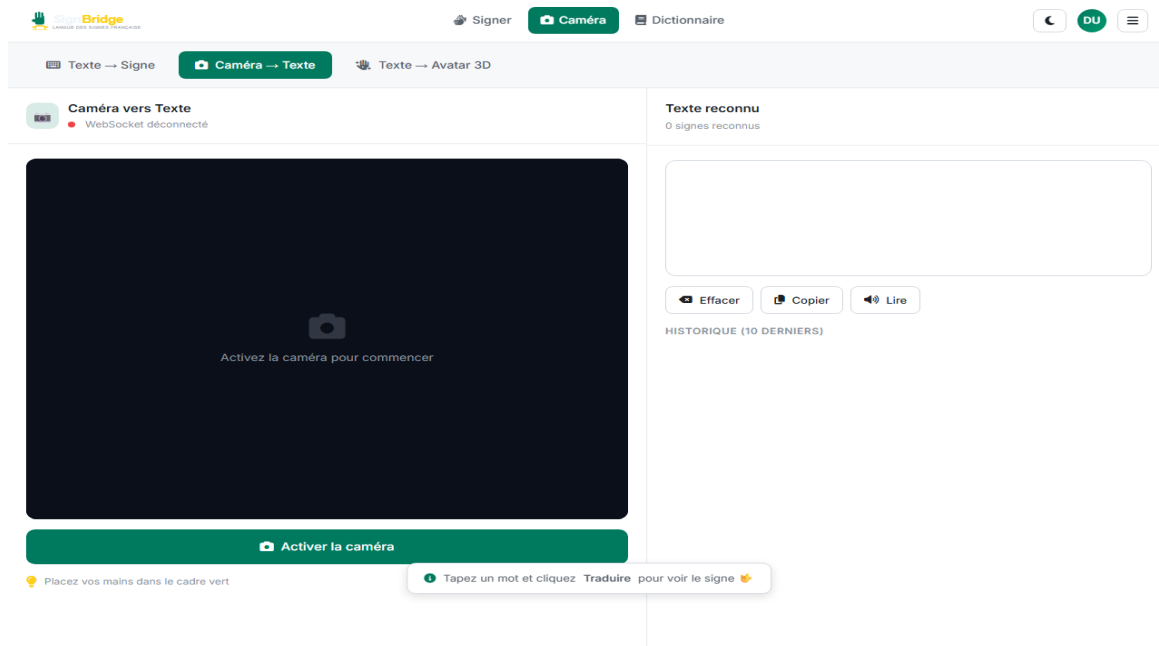


Figure 4 : Interface — Mode Caméra (Signe → Texte)

La reconnaissance en temps réel utilise un pipeline WebSocket bidirectionnel entre le navigateur et le serveur Django Channels. Voici le déroulement :

- L'utilisateur active la caméra — `getUserMedia` ouvre le flux vidéo
- MediaPipe Hands JS (WASM) traite chaque frame à ~12 fps
- Les 63 coordonnées sont envoyées via WebSocket (`ws://127.0.0.1:8001/ws/sign-recognition/`)
- Le consumer Django normalise les landmarks et appelle `predict_static()`
- Le Random Forest retourne le label et la confiance
- La réponse est retournée au navigateur en JSON
- Mécanisme de stabilisation : `STABLE_NEEDED = 3` frames consécutives identiques
- Seuil de confiance adaptatif : 0.22 (normal) → 0.11 après 6 frames stables

7.2 Mode Vidéo (upload)

- Upload XHR du fichier vidéo (MP4, AVI, MOV, WebM — max 100 Mo)
- `VideoProcessor.process_video()` decode la vidéo avec OpenCV
- Extraction des landmarks frame par frame via `LandmarkExtractor` (MediaPipe Python)
- Segmentation en fenêtres de 30 frames (chevauchement 15 frames)
- Classification de chaque segment via LSTM (`predict_dynamic()`)
- Seuil confiance > 0.6 pour valider un signe
- Retourne le texte reconstruit + segments détaillés

7.3 Paramètres de stabilisation

Paramètre	Valeur	Effet
STABLE_NEEDED	3 frames	Nb de prédictions identiques consécutives pour valider un signe

Paramètre	Valeur	Effet
CONF_THRESHOLD	0.22	Confiance minimale pour déclencher la validation
CONF_SUSTAINED	0.11	Seuil abaissé si le signe est stable depuis 6 frames
InMemoryChannelLayer	Dev uniquement	Layer de canal WebSocket (à remplacer par Redis en production)
FPS cible (vidéo)	10 fps	Sous-échantillonnage pour alléger le traitement vidéo

8. Entraînement du Modèle ML

8.1 Commande d'entraînement

Le modèle s'entraîne via la commande Django :

```
python manage.py train_model --algo all
```

Options : --algo rf (Random Forest seul), --algo lstm (LSTM seul), --algo all (les deux).

8.2 Pipeline d'entraînement — Random Forest

- 1. DatasetManager.load_dataset() charge tous les DatasetSample de la DB
- 2. Filtrage : qualite='BONNE', landmarks_data non null, longueur ≥ 63 floats
- 3. Construction de X (matrice $N \times 63$) et y (labels encodés)
- 4. Split train/test 80/20 avec stratification (train_test_split)
- 5. Entraînement : RandomForestClassifier(n_estimators=100, n_jobs=-1)
- 6. Évaluation : accuracy_score + classification_report
- 7. Sauvegarde : joblib.dump(model, 'signbridge_model/rf_model.pkl')
- 8. Enregistrement MLModelVersion en base (version, algorithme, accuracy, nb_classes)

8.3 Pipeline d'entraînement — LSTM

- 1. Mêmes DatasetSample chargés, groupés par signe
- 2. Normalisation des séquences à target_len=30 frames (padding/interpolation)
- 3. Architecture LSTM : couches LSTM (128 $\rightarrow$ 64 unités) $\rightarrow$ Dense (128 $\rightarrow$ nb_classes) $\rightarrow$ Softmax
- 4. Compilation : optimizer=Adam, loss=sparse_categorical_crossentropy
- 5. Entraînement : 50 epochs, batch_size=32, validation_split=0.2
- 6. Sauvegarde : model.save('signbridge_model/lstm_model.keras')

8.4 Entraînement via l'interface admin

L'API POST /api/admin/train/ lance l'entraînement en arrière-plan (thread daemon) sans bloquer le serveur. L'interface admin affiche les versions de modèles avec leur accuracy.

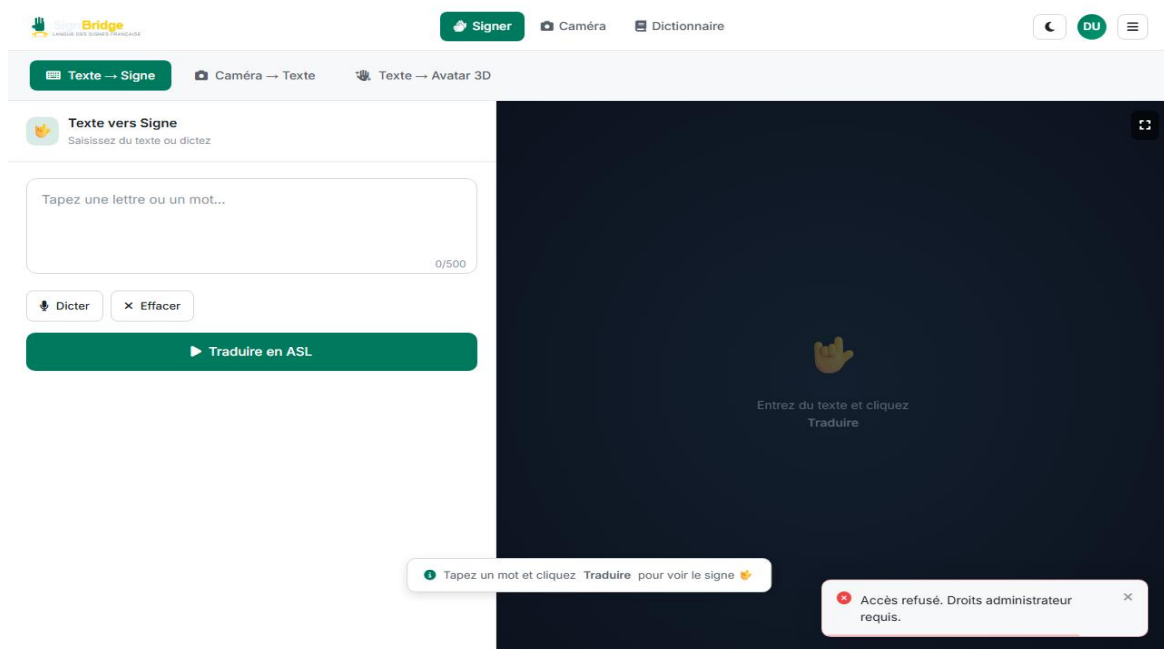


Figure 5 : Panel Administrateur — Gestion du dataset et entraînement

8.5 Composition du dataset (38 087 samples)

Source	Nb Samples	Signes couverts	Qualité coordonnées Z
GENERATED (synthétique)	14 400	A-Z + 0-9 (36)	3D artificiel (bruit gaussien)
KAGGLE (CSV)	20 834	A-Z + 0-9 (36)	Z=0 pour A-Z (2D), 3D pour 0-9
KAGGLE_IMG (MediaPipe réel)	2 853	0-9 uniquement	Vraie profondeur 3D
TOTAL	38 087	36 signes	—

9. Configurations du Projet

9.1 Fichier .env (variables d'environnement)

Variable	Valeur (dev)	Rôle
SECRET_KEY	(50 chars aléatoires)	Clé de signature Django (sessions, CSRF)
DEBUG	True	Mode debug Django — à mettre False en production
ALLOWED_HOSTS	127.0.0.1,localhost	Hôtes autorisés à accéder au serveur
DATABASE_URL	sqlite:///db.sqlite3	Chemin de la base SQLite
ML_MODEL_DIR	signbridge_model/	Répertoire des modèles ML sérialisés

9.2 Configuration Django (config/settings.py)

- AUTH_USER_MODEL = 'core.CustomUser' — modèle utilisateur personnalisé (login par email)
- ASGI_APPLICATION = 'config.asgi.application' — serveur ASGI Daphne/Channels
- CHANNEL_LAYERS : InMemoryChannelLayer (développement) — à remplacer par Redis en production
- STATICFILES_DIRS : inclut static/ et static_signs/ (animations GLB + images)
- STATICFILES_STORAGE : WhiteNoiseStorage avec compression Brotli/Gzip
- ML_MODEL_DIR = BASE_DIR / 'signbridge_model' — chemin des modèles

9.3 Configuration ASGI (config/asgi.py)

Le routeur ASGI dispatche les connexions entre HTTP (Django) et WebSocket (Channels) :

```
HTTP → django_asgi_app
ws/sign-recognition/ → SignRecognitionConsumer (WebSocket)
```

9.4 Démarrage du serveur

Développement :

```
python manage.py migrate
python manage.py setup_demo_data
daphne -p 8001 config.asgi:application
```

CSS (si node_modules manquant) :

```
npm install && npm run build:css
```

9.5 Paramètres du renderer 3D (hand3d_renderer.js)

Paramètre	Valeur	Rôle
Scale S	0.75	Facteur d'échelle des landmarks — réduit pour éviter le débordement du cadre
camera.position	(0, 0.25, 2.40)	Position de la caméra Three.js
camera.lookAt	(0, 0.25, 0)	Point de visée — centré sur la main
FOV	50°	Champ de vision vertical de la caméra perspective
tweenDur	450 ms	Durée de la transition entre deux poses (ease-in-out)

Paramètre	Valeur	Rôle
holdDur (normal)	900 ms	Durée d'affichage de chaque signe en vitesse normale
Pixel Ratio	min(devicePixelRatio, 2)	Résolution du rendu WebGL

10. Diagrammes UML

10.1 Diagramme de Cas d'Utilisation

Ce diagramme présente les interactions entre les trois types d'acteurs du système (visiteur non connecté, utilisateur connecté, administrateur) et les fonctionnalités offertes par SignBridge.

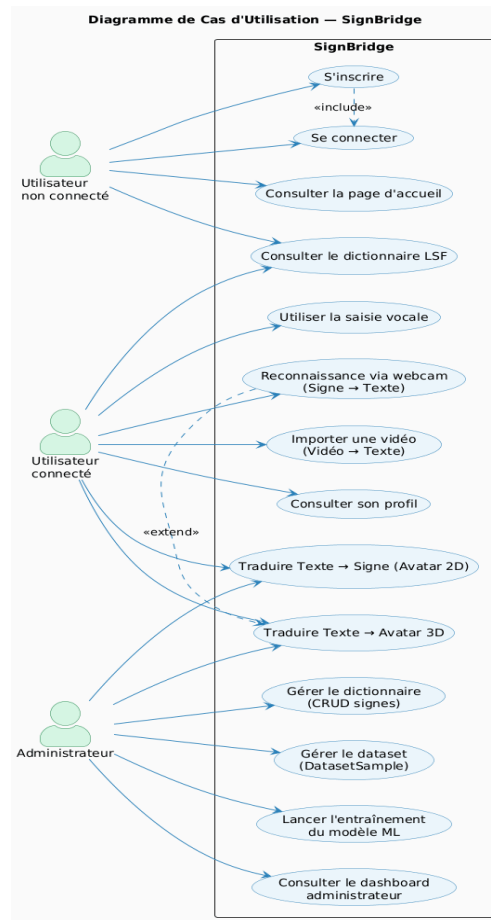


Figure 6 : Diagramme de Cas d'Utilisation — SignBridge

10.2 Diagramme de Séquence — Reconnaissance Webcam

Ce diagramme décrit le flux complet d'une session de reconnaissance gestuelle en temps réel, depuis la capture vidéo dans le navigateur jusqu'au retour du signe reconnu.

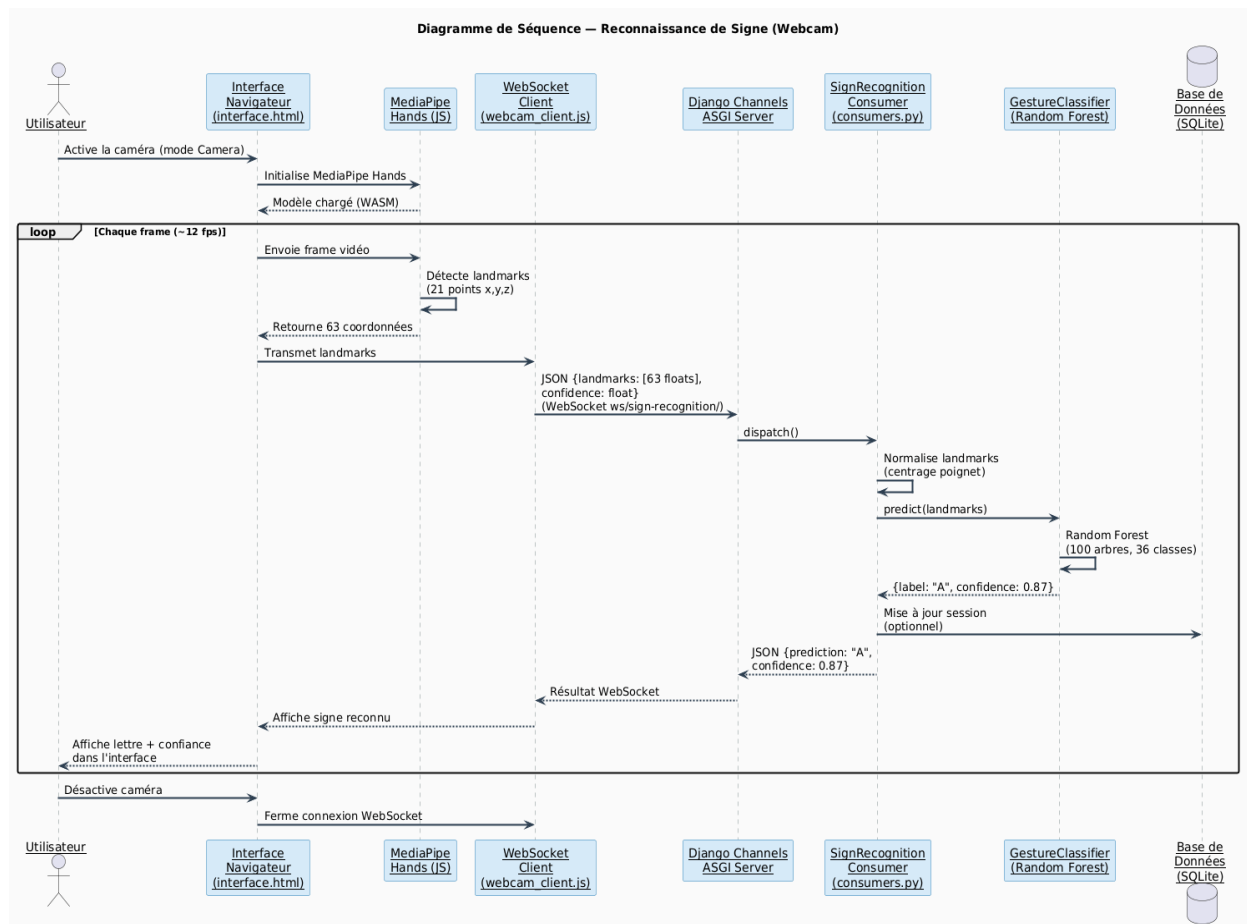


Figure 7 : Diagramme de Séquence — Mode Caméra (Signe → Texte)

10.3 Diagramme de Séquence — Texte vers Avatar 3D

Ce diagramme illustre le flux de traduction d'un texte vers l'animation de la main 3D, incluant l'initialisation du renderer et le chargement des poses depuis l'API.

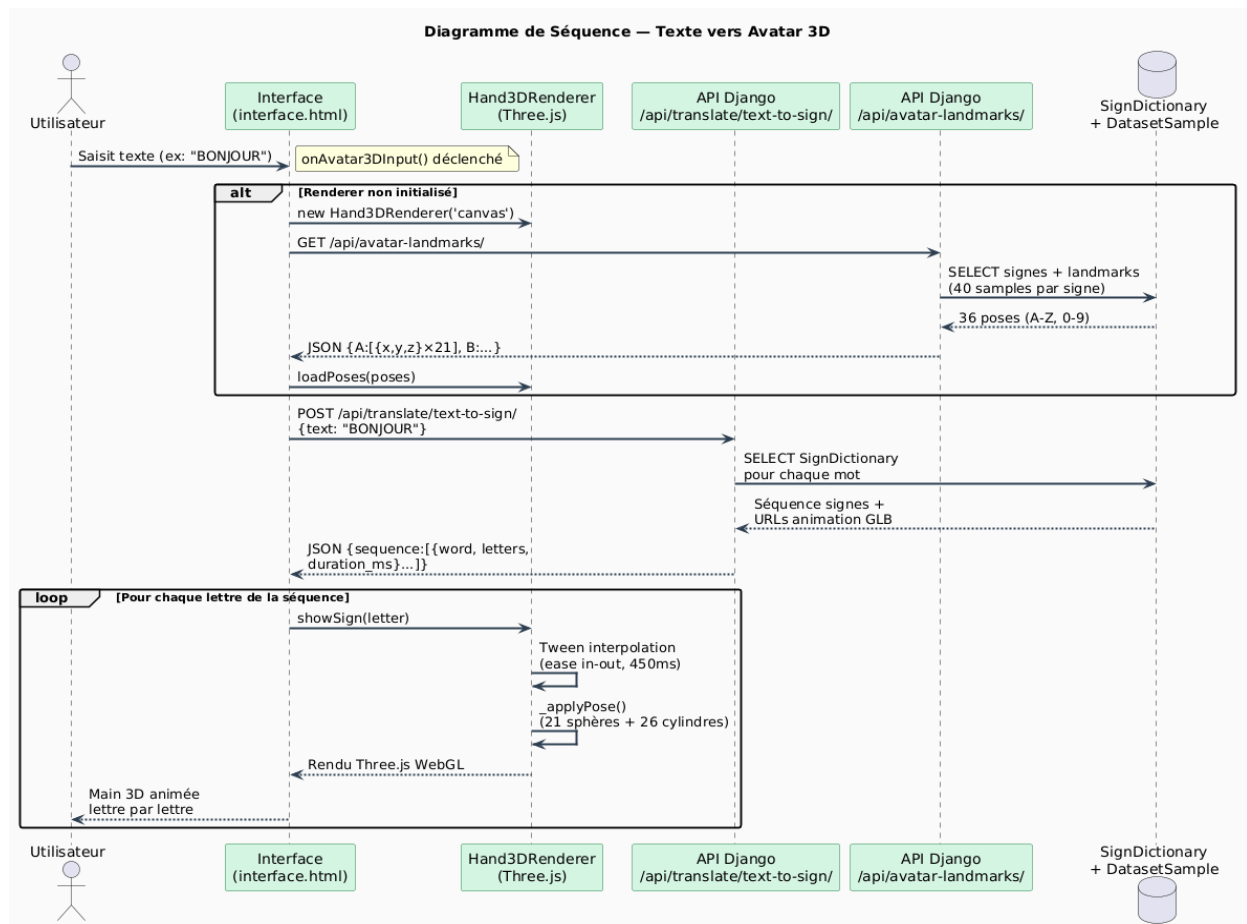


Figure 8 : Diagramme de Séquence — Mode Texte → Avatar 3D

10.4 Diagramme de Classes

Ce diagramme présente les classes principales du projet, leurs attributs, méthodes et relations. Il couvre à la fois les modèles Django (couche données) et les classes du moteur ML.

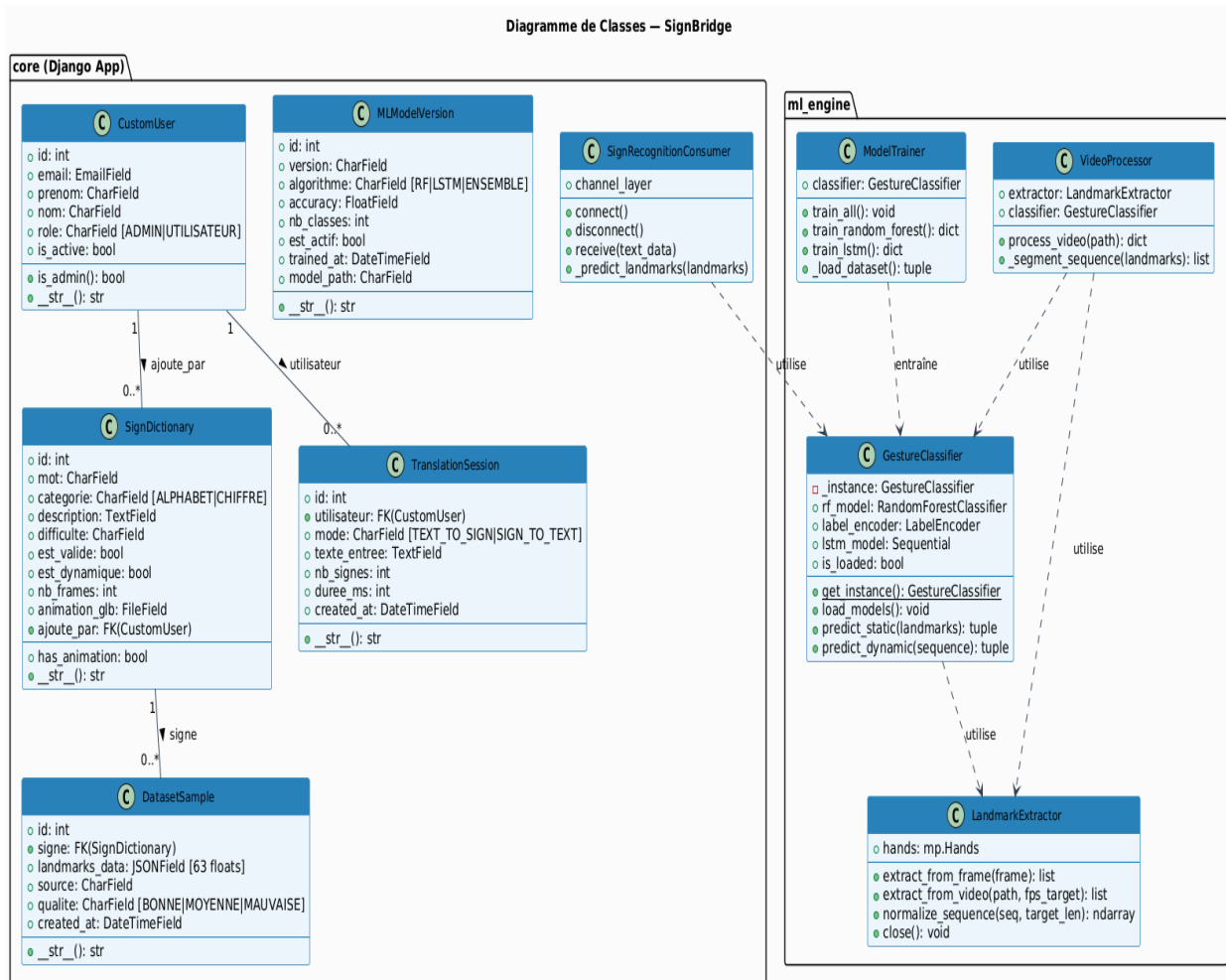


Figure 9 : Diagramme de Classes — Modèles Django + Moteur ML

10.5 Schéma Synoptique de l'Architecture

Ce schéma présente l'architecture complète en couches de SignBridge : présentation (navigateur), transport (HTTP/WebSocket), application (Django), ML/IA et données.

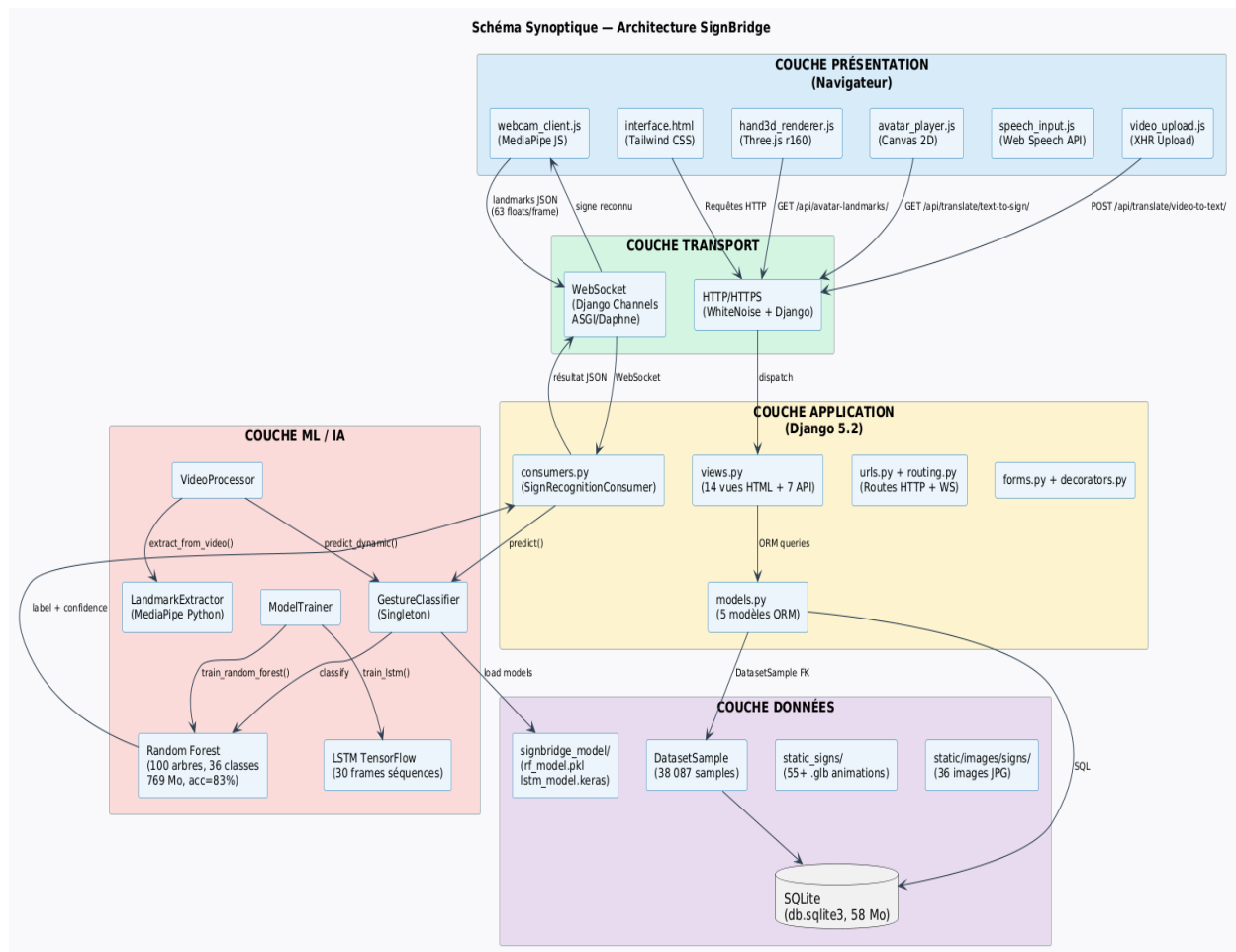


Figure 10 : Schéma Synoptique — Architecture globale SignBridge

11. Dictionnaire des Signes

Le dictionnaire de SignBridge recense 36 signes validés : les 26 lettres de l'alphabet (A-Z) et les 10 chiffres (0-9). Chaque entrée contient une description textuelle du geste, une image de référence, la catégorie (ALPHABET ou CHIFFRE), et un niveau de difficulté.

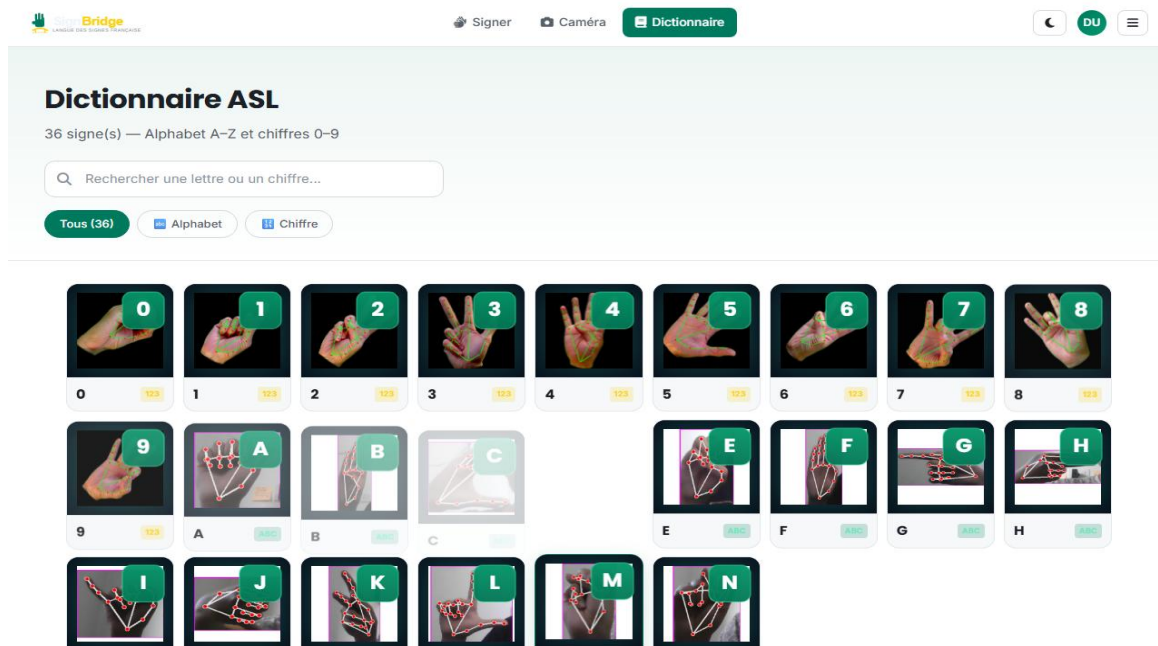


Figure 11 : Dictionnaire LSF/ASL — Vue d'ensemble des signes

11.1 Code PlantUML des diagrammes

Les diagrammes ont été générés via le serveur public PlantUML (plantuml.com). Le code source de chaque diagramme est inclus dans le fichier `doc_assets/generate_diagrams.py` du projet pour faciliter leur régénération ou modification.

```
python doc_assets/generate_diagrams.py
```

L'URL de génération suit le format :

```
https://www.plantuml.com/plantuml/png/{encoded_deflate_base64}
```

12. Conclusion

SignBridge est un projet full-stack ambitieux qui intègre avec succès plusieurs technologies d'intelligence artificielle dans une interface web moderne et accessible. Les points forts du projet sont :

- Une reconnaissance gestuelle fonctionnelle en temps réel (Random Forest, 83% de précision, 36 classes)
- Un pipeline WebSocket réactif basé sur Django Channels/Daphne pour la communication asynchrone
- Un renderer 3D procédural original en Three.js, sans recours à des assets 3D externes
- Une architecture modulaire claire séparant présentation, application, ML et données
- Une interface d'administration complète pour la gestion du dataset et l'entraînement

Les pistes d'amélioration identifiées sont :

- Capturer de vraies données LSF (Langue des Signes Française) avec des locuteurs natifs — les données actuelles sont ASL
- Remplacer InMemoryChannelLayer par Redis Channels en production pour la scalabilité
- Optimiser le chargement du modèle RF (769 Mo) via quantification ou compression
- Ajouter de vrais fichiers GLB avec animations pour le mode Texte→Signe (les .glb actuels sont des stubs vides)
- Mettre SESSION_COOKIE_SECURE=True et configurer un proxy HTTPS pour le déploiement production